data abstraction: Data abstraction is the process of hiding the complex details of a database system from the user and presenting only the necessary parts. It helps users to interact with the system without needing to understand its internal complexities.

Levels of Data Abstraction:

Physical Level (Lowest Level):

- Describes how data is physically stored in the database.
- Deals with complex data structures and storage details (e.g., file structures, indexing).
- Example: Data stored in blocks on a disk.

Logical Level (Middle Level):

- Describes what data is stored in the database and the relationships among the data.
- Hides the physical storage details from the users.
- Example: A university database has tables for students, courses, and enrollment.

View Level (Highest Level):

- Describes only part of the entire database.
- Provides different views to different users based on their needs.
- Example: A student can see only their grades, while an admin can see all students' records.

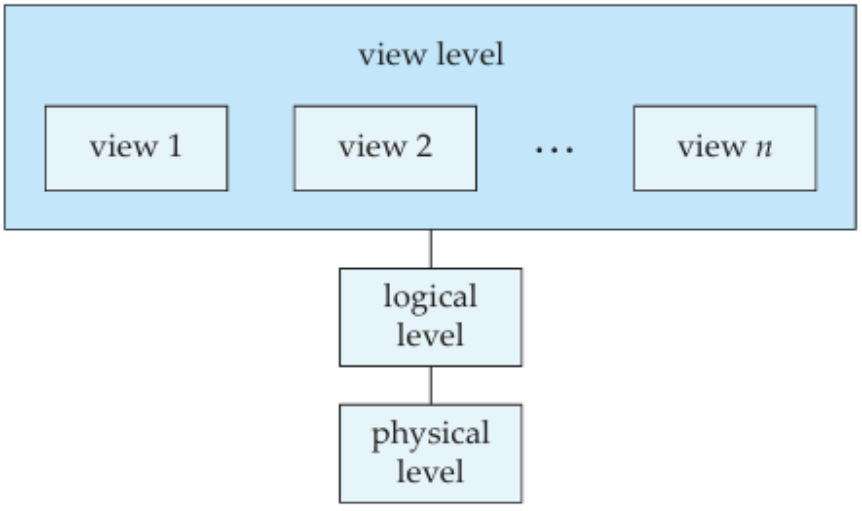


Figure 1.1 The three levels of data abstraction.

1. Schema:

Definition:

A **schema** is the **logical structure** or **blueprint** of a database. It defines how data is organized and how the relationships between data are managed.

Key Points:

- Describes tables, fields, data types, relationships, constraints, etc.
- Defined once and rarely changes.
- Acts like a **plan or design** of the database.

Example:
For a Student table, the schema might be:

Student(ID INT, Name VARCHAR, Age INT, Course VARCHAR)

This defines the structure of the Student table — its columns and data types.

---

2. Instance:

Definition:
An **instance** of a database refers to the **actual data** stored in the database at a particular moment in time.

Key Points:

- The instance **can change frequently** (as data is inserted, updated, or deleted).
- Represents the **state** of the database at a specific time.

Example:
Using the same Student schema, the instance could be:

ID	Name	Age	Course
101	Alice	20	B.Sc. CS
102	Bob	21	B.A. Eng

This is the **current data** in the table — i.e., an **instance**.

Data Models

A **data model** is a set of conceptual tools used to describe:

- **Data**
- **Relationships**
- **Semantics**
- **Constraints**

It helps in designing a database at **physical**, **logical**, and **view** levels.

Types of Data Models:

1. **Relational Model:**
    - Represents data using **tables (relations)**.
    - Each table has **rows (records)** and **columns (attributes)**.
    - Most widely used data model in modern databases.
    - Covered in Chapters 2–8.
2. **Entity-Relationship (E-R) Model:**
    - Uses **entities** and **relationships** to model data.
    - Entities are real-world objects.
    - Widely used in database design.
    - Covered in Chapter 7.
3. **Object-Based Data Model:**
    - Based on **object-oriented programming** concepts (Java, C++, C#).
    - Adds encapsulation, methods, and object identity to data modeling.

- **Object-relational model** combines object-oriented and relational features.

4. **Semistructured Data Model:**
    - Allows **flexible structure** where data items of the same type can have **different attributes**.
    - Unlike other models, it does not require a fixed schema.
    - **XML** is commonly used to represent this model.

DDL (Data Definition Language)

DDL commands are used to define or modify the structure of a database, including schemas, tables, indexes, and constraints. They do not manipulate the data itself but rather the database objects that hold the data.

Types of DDL commands:

- **`CREATE`**: Used to create database objects.
    - **Example**: `CREATE TABLE Students (StudentID INT PRIMARY KEY, FirstName VARCHAR(50), LastName VARCHAR(50));`
- **`ALTER`**: Used to modify the structure of an existing database object.
    - **Example**: `ALTER TABLE Students ADD Email VARCHAR(100);`
- **`DROP`**: Used to delete an entire database object.
    - **Example**: `DROP TABLE Students;`
- **`TRUNCATE`**: Used to remove all records from a table, including the space allocated for the records. It is a faster and more efficient command than `DELETE` for removing all records.
    - **Example**: `TRUNCATE TABLE Students;`
- **`RENAME`**: Used to rename a database object.
    - **Example**: `RENAME TABLE Students TO EnrolledStudents;`

🧩 Components of DDL:

Component	Description
Schema Definition	Defines the structure of tables, columns, and data types
Storage Definition	Specifies how data is stored and indexed (done using data storage language)
Constraint Definition	Ensures data validity and consistency
Authorization Specification	Manages user permissions for accessing and modifying data
Metadata Output	Generates metadata stored in the **data dictionary** (only accessible by DBMS)

🛡️ **Types of Constraints in DDL:**

Constraint Type	Description	Example
Domain Constraints	Ensure each attribute has values from a specified domain (e.g., integers, dates)	`budget numeric(12,2)`
Referential Integrity	Ensures foreign key values exist in the referenced table	Instructor's `dept_name` must exist in `department` table

Authorization	Controls who can **read**, **insert**, **update**, or **delete** data	A user may have only `read` and `insert` rights

---

🧠 Key Concept: Data Dictionary

- The **data dictionary** stores **metadata** (data about data).
- Updated automatically when DDL commands are executed.
- Not accessible by regular users—only by the **database system**.
- Used by the system to enforce constraints and optimize queries.

DML (Data Manipulation Language)

DML commands are used to manage data within schema objects. They are responsible for retrieving, inserting, deleting, and updating data in a database.

Types of DML commands:

- **`INSERT`**: Used to insert new data into a table.
  - **Example**: `INSERT INTO Students (StudentID, FirstName, LastName) VALUES (1, 'Alice', 'Smith');`
- **`UPDATE`**: Used to modify existing records in a table.

    - **Example**: `UPDATE Students SET Email = 'alice.smith@email.com' WHERE StudentID = 1;`
- **`DELETE`**: Used to remove existing records from a table.
    - **Example**: `DELETE FROM Students WHERE StudentID = 1;`
- **`SELECT`**: Used to retrieve data from one or more tables. This is often categorized as DQL (Data Query Language) but is also a core part of DML.
    - **Example**: `SELECT FirstName, LastName FROM Students WHERE StudentID = 1;`

🧩 Types of DML:

Type	Description	Example
1. Procedural DML	The user must specify **what data is needed** *and* **how to get it**	Writing loops or cursors in **PL/SQL**, or specifying join operations manually
2. Declarative DML	The user specifies **what data is needed**, but not how to get it	Using **SQL** (e.g., `SELECT name FROM students`)

🔹 **Declarative DML (SQL Example):**

```
SELECT name FROM employee WHERE department = 'HR';
```

- Retrieves names of employees in the HR department.
- User only specifies *what* data is needed.

🔹 **Procedural DML (PL/SQL-style logic):**

```
FOR employee IN (SELECT * FROM employee_table) LOOP
   IF employee.department = 'HR' THEN
      DBMS_OUTPUT.PUT_LINE(employee.name);
   END IF;
END LOOP;
```

- Specifies *how* to access and filter the data.

Term	Definition
Query	A statement that requests specific data from a database
Query Language	A language used to write queries, usually part of the DML (e.g., SQL)
Most Common	SQL (Structured Query Language)
Purpose	Data retrieval using conditions and table relationships

📘 Relational Database Concepts

✅ Table

A **table** is a collection of rows and columns used to represent data in a relational database. Each table stores data about a specific entity or relationship.

Example:
A `student` table might contain columns for student ID, name, age, and major.

```
+------------+------------+-----+----------+
```

```
| student_id | name       | age | major    |
+------------+------------+-----+----------+
| 1          | Alice      | 21  | Physics  |
| 2          | Bob        | 22  | Math     |
+------------+------------+-----+----------+
```

✅ Relation

A **relation** is essentially another term for a **table** in a relational database, comprising a set of tuples (rows) and attributes (columns).

Example:
The `student` table, as shown above, is a **relation** in a database.

✅ Tuple

A **tuple** is a single row of data in a relation (table). Each tuple contains values for each attribute (column) in the relation.

Example:
In the `student` table:

- The tuple `(1, Alice, 21, Physics)` represents a single student record.

✅ Attribute

An **attribute** is a column in a table, which represents a property or characteristic of the entity that the table is modeling.

Example:
In the `student` table, the attributes are `student_id`, `name`, `age`, and `major`.

✅ Relation Instance

A **relation instance** is a snapshot of a relation (table) at a particular moment in time. It represents the set of all tuples currently in the table.

Example:
At a given moment, the `student` table might contain two rows (instances). The **relation instance** is the set of those two rows:

```
{(1, Alice, 21, Physics), (2, Bob, 22, Math)}
```

✅ Domain

A **domain** is the set of all possible values that an attribute can take. It defines the permissible values for each attribute in a table.

Example:

- The `age` attribute might have a domain of integer values between 0 and 120.
- The `major` attribute might have a domain of values like `{Physics, Math, Computer Science, Biology}`.

✅ Atomic Domain

An **atomic domain** is a domain where each value in the set is indivisible, meaning each value is atomic or cannot be further decomposed.

Example:

- `age` is an atomic domain because each value represents a single integer (e.g., `21`, `22`).
- In contrast, an address like `123 Main St, City, ZIP` would not be atomic because it contains multiple components.

✅ Null Value

A **null value** represents missing or unknown data in a database. It indicates that the value for an attribute is not available or has not been specified.

Example:

- A `phone_number` attribute might have a `null` value if a student does not provide their phone number.

✅ Database Schema

The **database schema** is the **structure** that defines the organization of data within a database. It includes definitions for all the tables, attributes, relationships, and constraints.

Example:
The schema for a `student` database could include tables such as `students`, `courses`, and `enrollments`, with attributes like `student_id`, `name`, `age`, `course_id`, and `enrollment_date`.

✅ Database Instance

A **database instance** is a snapshot of the database at a particular point in time, including all the data in all tables and the state of the relations.

Example:
At a specific time, a database instance might contain records like:

```
students = {(1, Alice, 21, Physics), (2, Bob, 22, Math)}
courses = {(101, 'Math 101', 'Math'), (102, 'CS 101', 'Computer Science')}
```

✅ Relational Database

A **relational database** is a collection of **tables (relations)** that are related to each other through common attributes, often using keys.

Example:
A university database might contain multiple tables like `students`, `courses`, and `enrollments`, all related by attributes such as `student_id` and `course_id`.

✅ Keys

Superkey

A **superkey** is a set of one or more attributes that uniquely identify a tuple (row) in a relation (table). It can be larger than necessary, but it guarantees uniqueness.

Example:
In a `student` table, `{student_id}` is a superkey, as the `student_id` uniquely identifies each student. `{student_id, name}` is also a superkey, even though `student_id` alone would suffice.

Candidate Key

A **candidate key** is a minimal superkey, meaning it uniquely identifies a tuple in a relation and cannot have any redundant attributes.

Example:
In the `student` table, `{student_id}` is a candidate key, but `{student_id, name}` is **not** a candidate key, as `student_id` alone is sufficient to uniquely identify a student.

Primary Key

A **primary key** is a **candidate key** selected to uniquely identify tuples in a relation. Each table can have **one** primary key.

Example:
In the `student` table, `student_id` is the primary key because it uniquely identifies each student.

Primary Key Constraints

The **primary key constraint** ensures that the values in the primary key column(s) are unique and not null for each tuple in the relation.

Example:
In the `student` table, the constraint `PRIMARY KEY(student_id)` ensures that each student has a unique and non-null `student_id`.

✅ Foreign Key Constraint

A **foreign key** is an attribute in a relation that is used to link two tables. It refers to the primary key of another table and establishes a relationship between the two tables.

Referencing Relation

The **referencing relation** is the table that contains the foreign key. It refers to the primary key in another table.

Example:
In an `enrollments` table, the `student_id` attribute could be a foreign key that references the `student_id` primary key in the `students` table.

Referenced Relation

The **referenced relation** is the table that contains the primary key that is being referred to by the foreign key.

Example:
In the previous example, the `students` table is the **referenced relation** because it contains the `student_id` primary key, which is referenced by the `enrollments` table.

Query Language Types

Query languages can be categorized based on how users interact with them. Here are the main types:

1. Imperative Query Language

An **imperative** query language requires the user to specify both **what data** is needed and **how** to retrieve or manipulate that data. This means the user must provide detailed instructions on the operations to be performed.

- **Example**: Procedural languages like **Relational Algebra** or certain SQL commands (before optimizations).

- **Characteristics**:
    - Describes the sequence of operations.
    - Requires explicit steps to fetch or modify data.
    - Example operation: "Retrieve all students whose age is greater than 20."

2. Functional Query Language

A **functional** query language combines features of functional programming and relational databases. It focuses on **what data** is needed, typically using functions and expressions.

- **Example**: Languages like **SQL** when using higher-order functions or some functional languages (like **Datalog**).
- **Characteristics**:
    - Users describe what they need without detailing how to compute it.
    - The system determines the best way to execute queries.
    - Example operation: "Retrieve all student names who have enrolled in a course."

3. Declarative Query Language

A **declarative** query language focuses entirely on **what data** is needed, leaving the system to figure out the best way to retrieve or manipulate the data. The user specifies their intent, but the system decides the execution plan.

- **Example**: **SQL** is the most well-known declarative query language.
- **Characteristics**:
    - Easier for users, as they don't need to worry about the underlying execution.
    - Focuses on **what** should be retrieved or modified, rather than **how**.
    - Example operation: "Find all students who are over 20 years old."

Relational Algebra

Relational Algebra is a formal system for manipulating relations (tables). It provides a set of operations that take one or more relations as input and produce a new relation as output. Relational Algebra serves as a foundation for SQL and is an **imperative** query language because it defines a sequence of operations to be performed.

Relational-Algebra Expressions

A **Relational-Algebra expression** is a combination of operations that can be applied to one or more relations (tables). Here are the key operations in relational algebra:

1. Select (σ)

The **Select (σ)** operation is used to filter rows based on a specified condition (predicate). It extracts a subset of tuples (rows) from a relation.

- **Syntax**: `σ_condition(relation)`
- **Example**: `σ_age > 20(students)`
    - **Meaning**: Select all students whose age is greater than 20.

2. Project (Π)

The **Project (Π)** operation is used to select specific columns (attributes) from a relation. It effectively eliminates duplicate rows for the selected columns.

- **Syntax**: `Π_column1, column2,...(relation)`
- **Example**: `Π_name, major(students)`
    - **Meaning**: Project (select) the `name` and `major` columns from the `students` table.

3. Cartesian Product (×)

The **Cartesian Product** operation produces a relation that is the combination of every tuple from one relation with every tuple from another. If **R1** has **m** tuples and **R2** has **n** tuples, the result of **R1** × **R2** will have **m** × **n** tuples.

- **Syntax**: `R1 × R2`
- **Example**: `students × courses`
    - **Meaning**: Every student is paired with every course, producing a new relation with every possible combination of student and course.

4. Join (⋈)

The **Join (⋈)** operation is used to combine rows from two or more relations based on a related attribute, typically a foreign key matching a primary key.

- **Syntax**: `R1 ⋈ condition R2`
- **Example**: `students ⋈ students.student_id = enrollments.student_id enrollments`
    - **Meaning**: Join the `students` table with the `enrollments` table where the `student_id` matches in both tables.

5. Union (∪)

The **Union (∪)** operation combines the results of two relations, removing duplicates. Both relations must have the same number of attributes and the corresponding attributes must be of the same type.

- **Syntax**: `R1 ∪ R2`
- **Example**: `students ∪ graduates`
    - **Meaning**: Return all students and graduates in one relation, removing duplicates.

6. Set Difference (−)

The **Set Difference (−)** operation returns the tuples that are in one relation but not in another. The relations must have the same attributes.

- **Syntax**: `R1 - R2`
- **Example**: `students - graduates`
    - **Meaning**: Return all students who are not graduates.

7. Set Intersection (∩)

The **Set Intersection (∩)** operation returns the tuples that are common to both relations. The relations must have the same attributes.

- **Syntax**: `R1 ∩ R2`
- **Example**: `students ∩ graduates`
    - **Meaning**: Return all students who are also graduates.

8. Assignment (←)

The **Assignment (←)** operation allows the assignment of the result of a relational algebra expression to a temporary variable (or relation).

- **Syntax**: `X ← expression`
- **Example**: `Top_students ← σ_grade > 90(students)`
    - **Meaning**: Assign all students with a grade greater than 90 to the `Top_students` relation.

9. Rename (ρ)

The **Rename (ρ)** operation is used to rename a relation or its attributes. This is useful when performing complex queries to avoid confusion with attribute names.

- **Syntax**: `ρ(new_name, R)`
- **Example**: `ρ(students_2021, students)`
    - **Meaning**: Rename the `students` relation to `students_2021`.

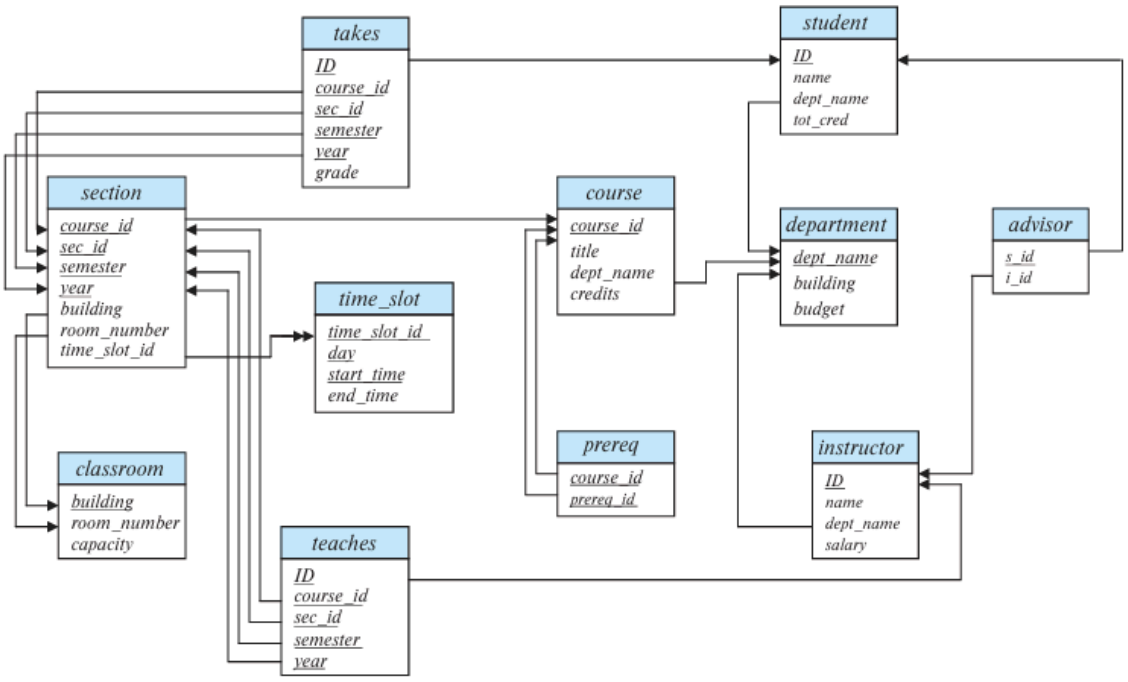


Figure 2.9 Schema diagram for the university database.

Functional Dependency (FD) – Simplified Definition:

In relational database design, a **functional dependency** (FD) is a constraint that helps us define relationships between attributes in a table. It ensures that the value of one set of attributes uniquely determines the value of another set.

Formal Definition:

- Given a relation schema **r(R)**, where **α** (a set of attributes) and **β** (another set of attributes) are subsets of **R**:

- A **functional dependency α → β** means that, for all pairs of tuples **t1** and **t2** in the table:

    - **If t1[α] = t2[α]**, then **t1[β] = t2[β]**.

In other words:

- If two rows have the same value for attributes in **α**, they must also have the same value for attributes in **β**.

What Does This Mean?

- **α (Determinant)**: The attributes that "determine" the values of other attributes.

- **β (Dependent)**: The attributes that depend on the values of **α**.

Example with a Student Table:

student_id	name	course_id	course_name
1	John	CS101	Database Systems

2	Alice	CS102	Data Structures
3	Bob	CS101	Database Systems

Functional Dependency:

- **student_id → name**

This means:

- If two rows have the same **student_id**, they must have the same **name**.

For example:

- For **student_id = 1**, the name must always be **John**.
- For **student_id = 2**, the name must always be **Alice**.

Real-World Explanation:

- **student_id → name**: Knowing the **student_id** is enough to know the **name**.
- If you have two students with the same **student_id**, they should have the same **name** in the table.

Why Functional Dependencies Matter:

- They help to define the structure of a database and ensure **data consistency**.
- They are used to **normalize** the database, eliminating unnecessary redundancy and preventing errors in data updates.

Functional Dependency Theory & Rules

Functional dependency theory provides a framework for reasoning about the constraints between attributes in a relational database. It defines how one set of attributes (the left-hand side) can uniquely determine another set of attributes (the right-hand side) in a relation.

Logical Implication and Closure (F+)

- **Logical Implication**:
    - A functional dependency **f** is logically implied by a set **F** of functional dependencies if every relation that satisfies **F** also satisfies **f**.
- **Closure of F (F+)**:
    - The closure **F+** is the set of all functional dependencies logically implied by **F**.
    - To compute **F+**, you apply inference rules until no new functional dependencies can be derived.

Armstrong's Axioms (Inference Rules)

Armstrong's Axioms are a set of three basic inference rules used to derive functional dependencies logically:

1. **Reflexivity**:
    - If **α** is a set of attributes and **β** is a subset of **α** (i.e., **β** $\subseteq$ **α**), then **α** $\rightarrow$ **β** holds.
    - **Example**: If **{student_id, name}** $\rightarrow$ **student_id**, then **{student_id}** $\rightarrow$ **student_id**.
2. **Augmentation**:
    - If **α** $\rightarrow$ **β** holds, and **γ** is any set of attributes, then **γα** $\rightarrow$ **γβ** holds.
    - **Example**: If **student_id** $\rightarrow$ **name**, then **{student_id, course_id}** $\rightarrow$ **{name, course_id}**.
3. **Transitivity**:

- If **α → β** and **β → γ** hold, then **α → γ** holds.
- **Example**: If **student_id → name** and **name → address**, then **student_id → address**.

Additional Derived Rules

These rules can be derived from Armstrong's Axioms and simplify finding implied dependencies:

1. **Union Rule**:
   - If **α → β** and **α → γ**, then **α → βγ**.
   - **Example**: If **student_id → name** and **student_id → course_id**, then **student_id → name, course_id**.
2. **Decomposition Rule**:
   - If **α → βγ**, then **α → β** and **α → γ**.
   - **Example**: If **student_id → name, course_id**, then **student_id → name** and **student_id → course_id**.
3. **Pseudotransitivity Rule**:
   - If **α → β** and **γβ → δ**, then **αγ → δ**.
   - **Example**: If **student_id → name** and **name, course_id → grade**, then **student_id, course_id → grade**.

A functional dependency is considered trivial if β is a subset of α (β ⊆ α); these are always satisfied by all relations

O

Normalization in Relational Database Design

Normalization is a process used in relational database design to minimize redundancy, avoid undesirable characteristics like update anomalies, and ensure data integrity. The process involves decomposing a database schema into smaller, well-structured relations (tables) while maintaining a lossless decomposition, meaning no information is lost when the decomposed relations are joined.

The primary goals of database design with respect to functional dependencies are:

1. BCNF: Achieving BCNF ensures the highest level of normalization, eliminating all redundancy and preventing data anomalies.

2. Losslessness: The decomposition of a relation into smaller relations should ensure that no information is lost and can be perfectly reconstructed using natural joins. This is crucial for preserving the integrity of the database.

3. Dependency Preservation: The decomposition should ensure that functional dependencies are not violated. Ideally, the decomposed relations should preserve all functional dependencies from the original schema.

Types of Normal Forms

Normalization uses functional dependencies to define several normal forms that a relational schema can conform to. These normal forms progressively eliminate redundancy and anomalies.

---

1. First Normal Form (1NF)

- **Definition**: A relation schema **R** is in **1NF** if all the attributes in **R** have atomic domains (indivisible values).

- **Explanation**: 1NF eliminates multivalued and composite attributes.

- **Example**:

- A table storing multiple phone numbers for an employee would not be in 1NF. To convert it into 1NF, each phone number must be stored in a separate tuple.

---

2. Second Normal Form (2NF)

- **Definition**: A relation schema **R** is in **2NF** if it is in **1NF** and if every non-prime attribute is fully functionally dependent on the entire candidate key.

- **Explanation**: 2NF eliminates **partial dependencies**, where a non-key attribute depends only on part of a composite key.

- **Example**:

  - Consider a relation with attributes **student_id**, **course_id**, and **grade**. If **student_id, course_id** is the primary key, but **grade** only depends on **course_id**, it violates 2NF because **grade** is partially dependent on **course_id**.

---

3. Third Normal Form (3NF)

- **Definition**: A relation schema **R** is in **3NF** if it is in **2NF** and no transitive dependency exists, i.e., if a non-prime attribute is dependent on another non-prime attribute.

- **Explanation**: 3NF eliminates **transitive dependencies**, where one non-key attribute depends on another non-key attribute through a third attribute.

- **Example**:

  - In a relation with attributes **employee_id**, **department_name**, and **department_head**, if **department_name → department_head**, then the schema is not in 3NF because **department_head** is transitively dependent on **employee_id** through **department_name**.

---

4. Boyce-Codd Normal Form (BCNF)

- **Definition**: A relation schema **R** is in **BCNF** if, for every functional dependency **α** → **β**, **α** is a superkey.

- **Explanation**: BCNF is stricter than 3NF. It eliminates cases where a non-superkey determines other attributes.

- **Example**:

    - In a table with attributes **course_id**, **instructor_id**, **instructor_name**, if **instructor_id** → **instructor_name** and **course_id** → **instructor_id**, the schema is not in BCNF because **instructor_id** is not a superkey.

---

5. Fourth Normal Form (4NF)

- **Definition**: A relation schema **R** is in **4NF** if, for every nontrivial multivalued dependency **α** →→ **β**, **α** is a superkey.

- **Explanation**: 4NF handles **multivalued dependencies** (MVDs), which are more general than functional dependencies and require additional steps for normalization.

- **Example**:

    - Consider a relation with **instructor_id**, **dept_name**, and **address**, where an instructor can belong to multiple departments and have multiple addresses. If these two facts are independent, **instructor_id** →→ **dept_name** and **instructor_id** →→ **address** are multivalued dependencies. If **instructor_id** is not a superkey, the relation is not in 4NF.

---

6. Higher Normal Forms (PJNF, DKNF)

- **Description**: These normal forms deal with more complex types of redundancy that cannot be eliminated using just functional and multivalued dependencies.

- **Project-Join Normal Form (PJNF)**: A stricter form of 5NF that handles **join dependencies**.

- **Domain-Key Normal Form (DKNF)**: A normal form that addresses more generalized constraints like domain constraints and key constraints.

==========

Semi-structured Data Definition and Model

Semi-structured data refers to data that does not have a rigid, predefined schema like traditional relational data. Instead, it possesses a flexible and evolving structure that allows individual data items of the same type to have different sets of attributes. This flexibility makes semi-structured data particularly useful for applications where the data structure changes frequently, such as web applications and user profiles that accumulate new attributes over time.

8.1.1.1 Flexible Schema

- **Wide Column Representation**: Some database systems support a flexible schema, where each record (tuple) may have a different set of attributes. This flexibility allows the schema to evolve without requiring changes to the entire database structure.

    - Example: A wide column representation allows each record to have varying attributes, and new attributes can be added as needed.

- **Sparse Column Representation**: A more restricted form of the wide column model involves a large but fixed set of attributes. Each tuple only uses the attributes it requires, and the remaining attributes are set to null.

    - Example: A sparse column representation might be used when only some attributes are relevant for each record.

8.1.1.2 Multivalued Data Types

- Many modern data representations allow attributes to store **non-atomic values**, such as sets, arrays, or multisets. This allows more natural modeling of complex

relationships.

    - **Sets**: Example: A user's interests may be stored as a set of topics, such as `{basketball, cooking, anime}`.

    - **Key-Value Maps**: Data can be stored as a collection of key-value pairs, where each key is unique. For example, e-commerce sites might represent product specifications as key-value maps: `{(brand, Apple), (ID, MacBook Air), (size, 13), (color, silver)}`.

- **Arrays**: Arrays are crucial for scientific and monitoring applications. For example, sensor data can be efficiently stored as an array of values, reducing overhead compared to storing individual tuples for each sensor reading.

    - **Array Databases**: Some databases, such as Oracle GeoRaster or SciDB, provide specialized support for storing and querying arrays.

8.1.1.3 Nested Data Types

- Semi-structured data representations can store **nested data types**, where an attribute is itself a composite structure, similar to the entity-relationship (E-R) model. This allows more complex data structures to be represented directly.

    - Example: A person's name might be represented as a nested structure with `firstname` and `lastname`.

- **JSON and XML** are two common representations that support nested data types. Both allow flexibility in how data is structured:

    - **JSON**: Commonly used for transmitting complex data between applications, such as mobile apps or web services.

    - **XML**: An older representation often used for configuration and data exchange between organizations.

8.1.1.4 Knowledge Representation

- **RDF (Resource Description Framework)** is an important representation for human knowledge, especially in the context of the web. It provides a simple, flexible way to store facts as triples.

- RDF triples consist of `(subject, predicate, object)`, which can represent entities and their relationships. RDF is well-suited for representing large-scale knowledge bases, like those in the semantic web, and is used in **knowledge graphs**.

- Unlike traditional relational models, RDF allows easy addition of new attributes and relationships without requiring schema changes.

Big Data: Definition, Characteristics, and Comparison with Traditional Databases

Big Data refers to vast amounts of data generated at high velocities, often in varied formats, which require specialized storage, processing, and analysis techniques. The challenges associated with Big Data have pushed the development of new technologies and paradigms for managing and analyzing data, with particular focus on **volume**, **velocity**, and **variety**.

Let's break down the key aspects of Big Data and contrast it with traditional databases.

Definition and Characteristics of Big Data

Big Data is generally defined by three main characteristics often referred to as the **Three Vs**:

1. Volume:

- **Definition**: Volume refers to the **enormous quantity of data** generated, often reaching petabytes (1,000 terabytes) and beyond. This is far larger than what traditional databases were originally designed to handle.

- **Examples**: The data generated by **social media platforms** (e.g., Facebook, Twitter) from millions of users, **internet of things (IoT)** devices generating real-time sensor data, or **e-commerce platforms** tracking customer activity.

- **Implication**: Due to the sheer size, traditional relational databases struggle to store and manage Big Data. Instead, Big Data systems use distributed storage across many machines, enabling horizontal scaling.

2. Velocity:

- **Definition**: Velocity refers to the **speed at which data is generated and must be processed**. This data is often generated in real-time or at high speeds, requiring systems that can ingest, store, and analyze it quickly.

- **Examples**: **Real-time stock market data**, **sensor data from autonomous vehicles**, or **social media feeds** that need to be analyzed in real-time to detect trends, anomalies, or to trigger immediate responses.

- **Implication**: Big Data platforms must support **streaming data processing** or handle **real-time analytics**. Systems like Apache Kafka, Apache Flink, and Apache Storm are often used for such applications.

3. Variety:

- **Definition**: Variety refers to the **different types and formats of data**. Big Data encompasses not only structured data (like rows in a table) but also semi-structured data (like JSON, XML), and unstructured data (like text, video, or images).

- **Examples**: **Social media posts** (text, images, video), **clickstream data**, and **log files** from web servers.

- **Implication**: Big Data platforms must be capable of handling **diverse data types** and adapt to changes in data formats over time. This requires flexibility that relational databases, which are designed for structured data, can't provide.

Summary: Key Differences Between Traditional Databases and Big Data

Feature	Traditional Databases	Big Data
Data Structure	Fixed schema (structured data)	Flexible schema (structured, semi-structured, unstructured data)
Volume	Small to medium (MB to GB)	Massive (TB to PB)
Scalability	Vertical scaling (single machine or limited clusters)	Horizontal scaling (many machines or clusters)
Querying	SQL (Structured Query Language)	MapReduce, Spark, NoSQL queries, and specialized frameworks

Consistency	ACID (Atomicity, Consistency, Isolation, Durability)	Eventual consistency, flexible models
Storage	Centralized storage (disks, SSDs)	Distributed storage (HDFS, NoSQL, key-value stores)
Use Cases	Transaction processing, data warehousing	Analytics, real-time processing, streaming, large-scale machine learning

Distributed File Systems (DFS): Key Concepts

A **distributed file system (DFS)** is a storage system that distributes files across multiple machines but presents a unified view to clients. This allows clients to access data as though it were stored on a single system, even though it may be spread across thousands of machines.

Why DFS are Needed:

1. **Handling Large Data Volumes (Volume)**: DFS enables storing and managing huge amounts of data (petabytes/exabytes) by partitioning files into blocks across many machines.

2. **Scalability**: DFS systems can **scale horizontally** by adding more machines, meeting the growing demands of large applications (e.g., social media, streaming services).

3. **Fault Tolerance and High Availability**: DFS replicate file blocks across multiple nodes to ensure data remains available even if some machines fail.

4. **Supporting Diverse Data Types (Variety)**: DFS can store unstructured data (videos, images, logs), which is challenging for traditional databases.

5. **Parallel Processing (Velocity)**: DFS enable **parallel I/O**, allowing fast data processing across multiple nodes, which is crucial for Big Data frameworks like **MapReduce**.

Examples of DFS:

1. **Google File System (GFS)**: Developed for large-scale applications, it divides files into blocks and replicates them for fault tolerance.

2. **Hadoop Distributed File System (HDFS)**: Open-source version of GFS, widely used for Big Data applications like **Apache Hadoop**.

3. **Amazon S3**: A cloud-based distributed object storage service, offering infinite scalability.

4. **Ceph**: Open-source distributed storage for high-performance and fault tolerance, supporting block, object, and file storage.

MapReduce: Overview and Need

MapReduce is a parallel processing paradigm designed for handling large-scale data by breaking it down into two key phases: **Map** and **Reduce**. It was popularized by **Google** and is commonly used in frameworks like **Hadoop** to process vast amounts of data efficiently.

Why MapReduce is Needed:

1. **Handling Big Data**:
    - Modern applications, like web services and social media, generate huge amounts of data. **MapReduce** helps process this data at scale, across many machines simultaneously.

2. **Parallel Processing**:
    - The **map()** function distributes the data processing across multiple machines. Each record is processed independently, leading to parallel computation.

3. **Simplified Fault Tolerance**:
    - In large systems, machine failures are inevitable. **MapReduce** handles this by automatically re-executing failed tasks, ensuring data integrity without requiring the programmer to manage failure recovery manually.

4. **Abstraction of Complexity**:

    - Programming parallel data processing and handling complex distributed systems can be difficult. **MapReduce** abstracts away the complexity of parallelism and fault tolerance, allowing developers to focus only on the logic of **map()** and **reduce()** functions, rather than on the intricacies of distributed systems.

5. **Scalability**:

    - **MapReduce** is highly scalable, allowing data processing to grow from a single machine to thousands, making it ideal for large datasets and cloud-based environments.

MapReduce Example: Word Count and Log Processing

1. Word Count Application:

MapReduce is often used for counting word frequencies in large datasets, such as documents or text inputs.

Steps:

- **Map Function:**
  The `map()` function processes each line of text, splits it into words, and emits key-value pairs (word, 1).
  Example Input:
  `"One a penny, two a penny, hot cross buns."`
  Output:
  `("one", 1), ("a", 1), ("penny", 1), ("two", 1), ("a", 1), ("penny", 1), ("hot", 1), ("cross", 1), ("buns", 1)`

- **Shuffle and Sort:**
  The system groups records by key (word) and sorts them.
  Example Output (sorted):
  `("a", [1, 1]), ("buns", [1]), ("cross", [1]), ("hot", [1]), ("one", [1]), ("penny", [1, 1]), ("two", [1])`

- **Reduce Function:**
  The `reduce()` function sums the counts for each word.

Example Output:

```
("one", 1), ("a", 2), ("penny", 2), ("two", 1), ("hot", 1), ("cross", 1), ("buns", 1)
```

---

2. Log Processing Example:

In log processing, we count how many times a file was accessed in a given directory within a time range.

Steps:

- **Map Function:**
  The `map()` function processes each log entry, extracting the date, time, and filename. It emits `(filename, 1)` if the date is within the range and the file is in the target directory.
  Example Input (log entry):
  `"2013/02/21 10:31:22 /slide-dir/11.ppt"`
  Output:
  `("slide-dir/11.ppt", 1)`

- **Shuffle and Sort:**
  The system groups all pairs by filename, and the values (counts) are aggregated.
  Example Output:

```
("slide-dir/11.ppt", [1, 1]), ("slide-dir/12.ppt", [1])
```

- **Reduce Function:**
  The `reduce()` function sums the counts for each filename.
  Example Output:
  `("slide-dir/11.ppt", 2), ("slide-dir/12.ppt", 1)`

Parallel Processing in MapReduce:

- **Map Tasks:**
  Data is partitioned and each partition is processed by a different machine in

parallel.

- **Reduce Tasks:**
  After the shuffle and sort, reduce tasks run in parallel to aggregate the results.

- **Distributed Workflow:**

    - **Input Partitioning:** Split data into parts.

    - **Map Tasks:** Each part is processed on different nodes.

    - **Shuffle and Sort:** Grouped data is shuffled and sorted by key.

    - **Reduce Tasks:** Results are aggregated by reduce functions across multiple nodes.

MapReduce in Hadoop:

- **Hadoop:**
  Hadoop is an open-source implementation of MapReduce. It uses Java classes for the `Mapper` and `Reducer` functions.

- **Distributed File System (HDFS):**
  Hadoop stores data across multiple machines using the Hadoop Distributed File System (HDFS).

- **Combiner Function:**
  Hadoop allows the use of a Combiner function during the map phase to perform partial aggregation, reducing the amount of data transferred to the reduce phase.

Classification Algorithms:

1. **Decision Tree Classifiers:**

    - **Description:** A decision tree uses a tree-like structure where each internal node represents a condition or decision based on an attribute, and each leaf node represents a class label. The classification decision is made by

traversing the tree based on attribute values.

- **Example Use:** Classifying creditworthiness based on attributes like income and education level (e.g., "good," "average," "bad" credit risk).

2. **Bayesian Classifiers:**

   - **Description:** Bayesian classifiers use Bayes' Theorem to calculate the probability of an instance belonging to a particular class based on the distribution of attributes for each class. They often assume that attributes are independent, known as "naive Bayes."

   - **Example Use:** Predicting the creditworthiness of an applicant based on income and other features.

3. **Support Vector Machine (SVM) Classifiers:**

   - **Description:** SVM classifiers create a hyperplane that separates data points of different classes, ensuring the maximum margin between the closest points of each class. SVMs can handle linear and non-linear classification by using kernel functions.

   - **Example Use:** Classifying customer credit risk (e.g., "good" or "bad") based on various attributes like income and debt levels.

4. **Neural Network Classifiers:**

   - **Description:** Neural networks consist of interconnected layers of "neurons" that process input data to predict a class label. The weights of connections are learned during training via backpropagation.

   - **Example Use:** Classifying large datasets, such as in vision tasks (e.g., object recognition) or financial predictions like customer credit risk.

Clustering Algorithms:

1. **Clustering Based on Distance Metrics:**

- **Description:** Clustering algorithms group data points into clusters so that the average distance of points from the centroid (or center) of their cluster is minimized. The exact method can vary based on how the distance between points is defined.

- **Example Use:** Finding clusters of customers with similar buying behavior to suggest associated products.

2. **Hierarchical Clustering:**

   - **Description:** Hierarchical clustering groups data into a tree-like structure where smaller clusters are nested within larger ones. This approach can be agglomerative (bottom-up) or divisive (top-down).

   - **Example Use:** Categorizing species in biology (e.g., grouping species under broader classes like "mammalia" or "reptilia").

3. **Clustering for Collaborative Filtering:**

   - **Description:** In this specific application of clustering, users or items (e.g., movies, books) are clustered based on preferences. The preferences of similar users are used to predict new interests or recommendations.

   - **Example Use:** Recommending movies to users based on their past preferences and clustering with similar users.

A data warehouse is a repository (or archive) of information gathered from multi ple sources, stored under a unified schema, at a single site. Once gathered, the data are stored for a long time, permitting access to historical data. Thus, data warehouses pro vide the user a single consolidated interface to data, making decision-support queries easier to write. Moreover, by accessing information for decision support from a data warehouse, the decision maker ensures that online transaction-processing systems are not affected by the decision-support workload

cation, and they perform queries only on data that are old enough that they have

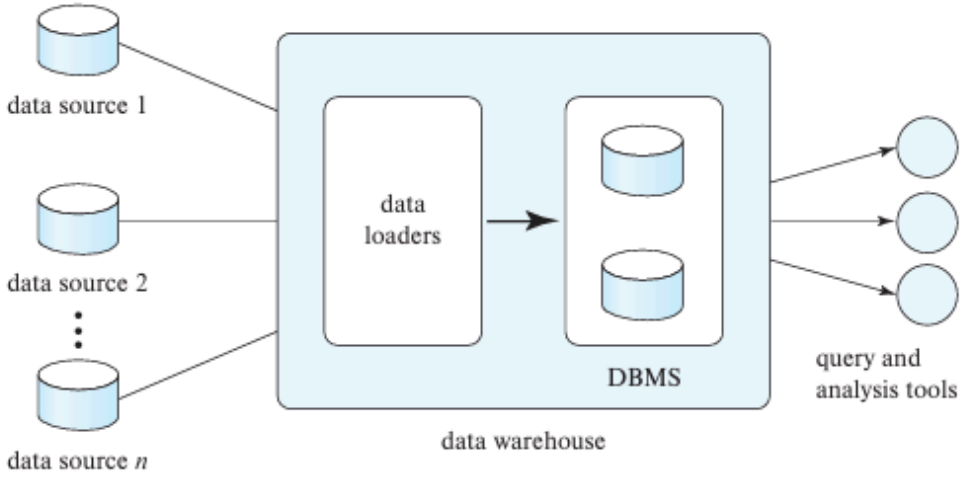


Figure 11.1 Data-warehouse architecture.

Data Warehouse Components

1. **Data Gathering**: Continuous or periodic data extraction from various sources, with considerations for timeliness and freshness.

2. **Schema Integration**: Harmonizing data from diverse sources into a consistent schema for easier analysis.

3. **Data Transformation and Cleansing**: Correcting inconsistencies, eliminating duplicates, and transforming data to fit the warehouse schema.

4. **Propagating Updates**: Ensuring that updates in source systems are accurately reflected in the data warehouse.

5. **Data Summarization**: Storing aggregated data to optimize querying performance.

6. **ETL vs. ELT**: Two different approaches for handling data extraction, transformation, and loading.

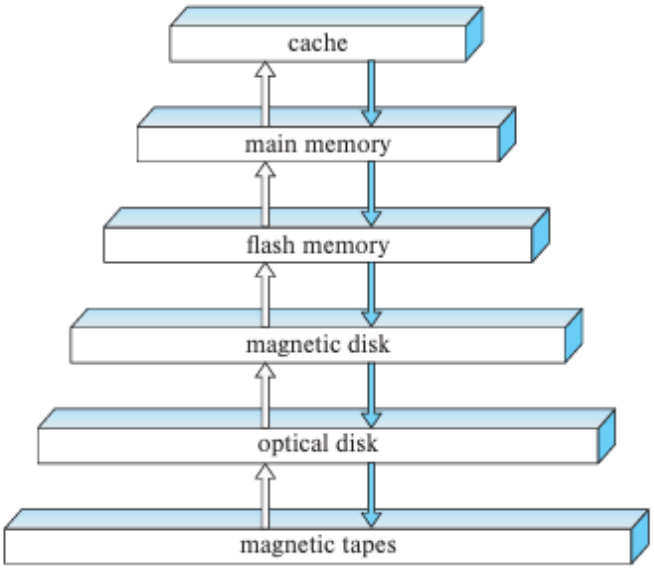


Figure 12.1 Storage device hierarchy.

RAID's main advantage is increased performance, reliability, and redundancy by combining multiple drives into a single logical unit, while its disadvantages include lower usable capacity, higher cost, and the lack of protection against corruption or user error. Not all RAID levels offer the same balance; RAID 0 prioritizes speed but offers no protection, whereas RAID 1 provides redundancy at the cost of capacity.

Advantages of RAID

Increased Performance:
Data can be spread across multiple drives (striping), allowing for faster read and write speeds.
Data Redundancy and Fault Tolerance:
Many RAID levels can protect against data loss by creating copies of data or using parity information, so the system can continue to operate even if a drive fails.
Improved Availability:
With proper configurations, RAID can offer protection against downtime caused by a single drive failure, allowing for hot-swapping of failed drives.

Disadvantages of RAID

Reduced Usable Capacity:
Some RAID levels (like RAID 1) require drives to be duplicated, effectively halving the usable storage space compared to the total physical disk capacity.
Higher Cost:
More drives are needed to achieve a certain storage capacity, which increases the overall hardware cost.
Complexity:
While software RAID is cheaper, hardware RAID is more expensive.

Not a Backup:
RAID does not protect against data corruption, accidental deletions, or malicious attacks; a separate backup strategy is essential.
Potential for Slower Writes:
Some RAID levels, such as RAID 5 and RAID 6, can experience slower write performance due to the overhead of calculating parity information.

RAID 0 (Striping): Data is split across multiple disks without redundancy. It improves performance but offers no fault tolerance.

RAID 1 (Mirroring): Data is mirrored across disks, providing redundancy. Some vendors refer to RAID 10 as striping + mirroring, while RAID 1 alone is just mirroring without striping.

RAID 5 (Distributed Parity): Data and parity information are spread across all disks, allowing for fault tolerance with a single disk failure. Parity blocks are distributed to avoid a single point of failure.

RAID 6 (Dual Parity): Similar to RAID 5 but stores additional parity information (Q), enabling it to tolerate two disk failures using error-correcting codes like Reed-Solomon.

Variants & Nested RAID: Vendors may combine levels (e.g., RAID 10) or create nested schemes, such as stripe across multiple RAID arrays for increased performance and reliability.

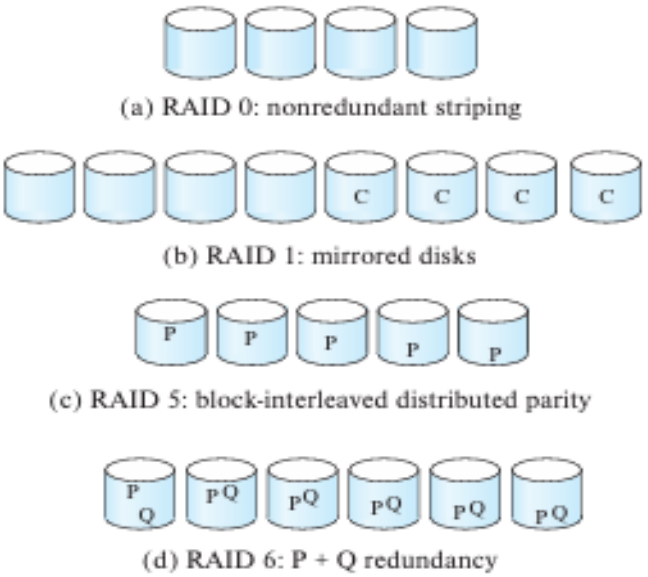


Figure 12.4 RAID levels.

Choice of RAID Level
The factors to be taken into account in choosing a RAID level are:

• Monetary cost of extra disk-storage requirements.
• Performance requirements in terms of number of I/O operations per second.
• Performance when a disk has failed.
• Performance during rebuild (i.e., while the data in a failed disk are being rebuilt on a new disk).

⚖️ Indexing vs Hashing: Key Differences

Feature	Indexing	Hashing
Goal	Fast data retrieval (range or exact match)	Fast exact match lookup
Mechanism	Tree-based structures (B-Tree, etc.)	Hash functions + hash tables
Supports	Range queries, sorting	Only exact matches
Ordering	Maintains order (in some types like B-tree)	Does not maintain order
Performance	Good for read-heavy workloads	Excellent for direct lookups
Collisions	Not applicable (uses tree)	Needs collision handling
Example	SQL: `CREATE INDEX ON name;`	Key-value stores, hash maps

factors should be considered when choosing an indexing technique

There are two basic kinds of indices:

• **Ordered indices**: Based on a sorted ordering of the values.

• **Hash indices**: Based on a uniform distribution of values across a range of buckets. The bucket to which a value is assigned is determined by a function, called a **hash function**.

We shall consider several techniques for ordered indexing. **No one technique is the best. Rather, each technique is best suited to particular database applications.** Each technique must be evaluated on the basis of these factors:

• **Access types**: The types of access that are supported efficiently. Access types can include finding records with a specified attribute value and finding records whose attribute values fall in a specified range.

• **Access time**: The time it takes to find a particular data item, or set of items, using the technique in question.

• **Insertion time**: The time it takes to insert a new data item. This value includes the time it takes to find the correct place to insert the new data item, as well as the time it takes to update the index structure.

• **Deletion time**: The time it takes to delete a data item. This value includes the time it takes to find the item to be deleted, as well as the time it takes to update the index structure.

• **Space overhead**: The additional space occupied by an index structure. Provided that the amount of additional space is moderate, it is usually worthwhile to sacrifice the space to achieve improved performance.

Indexing and Hashing Definitions

1. **Indexing:**
    - **Ordered indices**: These are based on a sorted ordering of values in a database.
    - **Hash indices**: These rely on a hash function to map data to buckets, ensuring a uniform distribution of values across them.
2. **Hashing:**

- The hash function assigns each key to a specific bucket, ensuring that records are evenly distributed across available buckets. There are two types of hashing:
    - **Static Hashing**: The number of buckets is fixed, which may cause overflow if the number of records exceeds the number of buckets.
    - **Dynamic Hashing**: Involves incrementally increasing the number of buckets to accommodate more records.

Types of Indices

1. **Ordered Indices**: Based on sorting, efficient for range queries.
2. **Hash Indices**: Based on a hash function, efficient for equality queries but not range queries.

B-tree and B+-tree

1. **B-tree**:
    - A balanced tree structure where each node can have multiple children, making it efficient for disk-based indexing.
    - **B-tree nodes** contain search keys and pointers to subtrees or records, which helps in narrowing down the search.
2. **B+-tree**:
    - A specialized version of the B-tree where all records are stored in the leaf nodes, and the internal nodes store only keys for navigation.
    - **Leaf nodes** store the actual records, while **nonleaf nodes** only store keys for navigation. This makes it suitable for both sequential and random access.

- A **B+-tree** provides more efficient range queries than the standard **B-tree**.

Differences between B-tree and B+-tree

1. **Storage of Data**:
   - **B-tree**: Data can be stored in both leaf and nonleaf nodes.
   - **B+-tree**: Data is only stored in leaf nodes, nonleaf nodes store pointers.
2. **Search Efficiency**:
   - **B-tree**: Data might be found before reaching the leaf node.
   - **B+-tree**: Only the leaf nodes contain the actual data, so search always reaches the leaf node.
3. **Range Queries**:
   - **B-tree**: Less efficient in handling range queries.
   - **B+-tree**: Optimized for range queries as leaf nodes are linked sequentially.
4. **Fan-out**:
   - **B-tree**: Lower fan-out compared to B+-tree, leading to a taller tree.
   - **B+-tree**: Higher fan-out due to the larger number of keys per node, resulting in a shorter tree.

Dense and Sparse Indices An index entry,orindex record, consists of a search-key value and pointers to one or more records with that value as their search-key value. The pointer to a record consists of the identifier of a disk block and an offset within the disk block to identify the record within the block.

There are two types of ordered indices that we can use:

• Dense index: In a dense index, an index entry appears for every search-key value in the file. In a dense clustering index, the index record contains the search-key value and a

pointer to the first data record with that search-key value. The rest of therecordswiththesamesearch-keyvaluewouldbestoredsequentiallyafterthe f irst record, since, because the index is a clustering one, records are sorted on the same search key. In a dense nonclustering index, the index must store a list of pointers to all records with the same search-key value.
• Sparseindex:Inasparse index, an index entry appears for only some of the search key values. Sparse indices can be used only if the relation is stored in sorted order of the search key; that is, if the index is a clustering index. As is true in dense indices, each index entry contains a search-key value and a pointer to the first data record with that search-key value. To locate a record, we find the index entry with the largest search-key value that is less than or equal to the search-key value for which we are looking. We start at the record pointed to by that index entry and follow the pointers in the file until we find the desired record.

Spatial Data Definition:

Spatial data refers to information that identifies the geographic location and shape of natural or constructed features and boundaries on the earth's surface. It encompasses any data that has a geographic or geometric component, such as points, lines, and polygons, which define objects in a two-dimensional or multi-dimensional space.

- **Example:** The location of a restaurant is represented by a **(latitude, longitude)** pair. Similarly, a farm or lake can be represented by a **polygon** with coordinates for each corner.

Temporal Data Definition:

Temporal data, in contrast, refers to data associated with a specific time period. It typically includes time intervals that specify the validity or occurrence of an event or fact. A time interval has a start time and an end time, and may be open or closed at both ends. Temporal data helps in tracking the changes that occur over time.

- **Example:** A course identifier may have different titles at different times. A tuple with the course identifier and title will have an associated **valid time period** (start time and end time).

Key Differences Between Spatial and Temporal Data:

Aspect	Spatial Data	Temporal Data
Definition	Data that refers to a point or region in two or more dimensions.	Data that is associated with a time period or time interval.
Example	(Latitude, Longitude), Polygons (e.g., a lake, a building).	(Course ID, Title, Validity Period), (Employee ID, Salary, Date Range).
Data Representation	Points, lines, polygons, and other shapes in geometric space.	Time intervals with start and end times, which may be open or closed.
Primary Purpose	To represent and query physical locations and boundaries.	To model and track changes over time for a particular entity.
Query Types	Range queries, nearest neighbor queries, spatial joins.	Time-based queries, such as retrieving data valid at a specific time or within a range.
Indexing Structures	R-tree, k-d tree, quadtrees, and k-d-B trees.	B+-tree, interval B+-tree, R-tree (for time-bound objects), and temporal indexing.
Challenges	Efficiently handling multi-dimensional range queries and nearest neighbor searches.	Efficient indexing of intervals, handling infinite time values, and managing overlapping time intervals.
Storage	Stored as points, lines, or polygons with bounding boxes.	Stored with time intervals, potentially including infinite end times.

Spatial Data Indexing Techniques (from the text):

1. **k-d Tree**: A spatial indexing structure for multi-dimensional data where each level partitions the space along a specific dimension (cycling through the dimensions). It efficiently supports range queries by recursively partitioning the data into smaller subspaces.

2. **R-tree**: A balanced tree structure useful for indexing objects that span regions of space such as line segments, rectangles, and polygons. Internal nodes store

bounding boxes for subtrees, while leaf nodes store the indexed objects themselves. R-trees are suitable for spatial data with regions and complex shapes.

3. **k-d-B Tree**: An extension of the k-d tree that allows multiple child nodes for each internal node, making it more efficient for secondary storage and handling larger datasets than the basic k-d tree.

Temporal Data Indexing Techniques (from the text):

1. **B+-tree for Temporal Data**: A B+-tree index can be used to index temporal data by associating tuples with both the attribute value and the start time. This index can support queries based on the current or historical validity of tuples.

2. **R-tree for Temporal Data**: The R-tree can be adapted to index temporal data by treating the time intervals as one of the dimensions. For example, the valid time interval of a tuple (e.g., employee salary over time) can be treated as a "spatial" dimension in addition to the regular attribute dimension.

3. **Interval B+-tree**: This specialized index is designed for indexing intervals in a single dimension (such as time) and can offer more efficient query handling compared to R-trees for temporal data.

Challenges in Indexing Spatial and Temporal Data:

- **Spatial Data**: The complexity lies in efficiently querying multi-dimensional spaces and handling range queries (such as finding all points within a region) or nearest neighbor searches. Structures like **R-trees** are often used but can suffer from inefficiencies when objects cross partitioning lines.

- **Temporal Data**: Temporal data introduces the challenge of managing intervals, especially those with **infinite end times** (indicating ongoing validity). While **B+-trees** are commonly used for temporal indexing, more specialized structures like **Interval B+-trees** or hybrid spatial-temporal indices may be used to handle the complexities of time-based queries efficiently.

Combining Spatial and Temporal Data Indexing:

For systems that need to support both spatial and temporal queries, an R-tree can be adapted to index data with **spatial and temporal components**. In this case:

- The **spatial dimension** can be one attribute (e.g., location coordinates).
- The **temporal dimension** can represent the valid time interval of each object, essentially forming a "time-bound object" for spatial-temporal queries.

Such an index allows for efficient querying of data that is valid over a specific time period, like finding all objects in a region that were valid during a specific time frame.

Query processing refers to the range of activities involved in extracting data from a database. The activities include translation of queries in high-level database languages into expressions that can be used at the physical level of the file system, a variety of query-optimizing transformations, and actual evaluation of queries.

The steps involved in processing a query appear
The basic steps are:
1. Parsing and translation.
2. Optimization.
3. Evaluation.

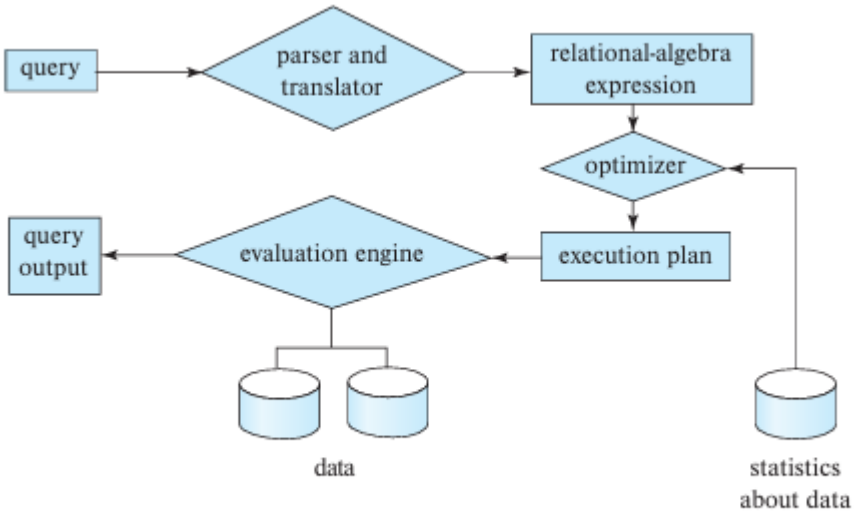


Figure 15.1 Steps in query processing.

Query optimization is the process of selecting the most efficient query-evaluation plan fromamongthemanystrategiesusuallypossibleforprocessingagivenquery,especially if the query is complex.

A transaction is a unit of program execution that accesses and possibly updates various data items. Usually, a transaction is initiated by a user program written in a high-level data-manipulation language (typically SQL), or programming language (e.g., C++ or Java), with embedded database accesses in JDBC or ODBC. A transaction is delimited by statements (or function calls) of the form begin transaction and end transaction.The transaction consists of all operations executed between the begin transaction and end transaction.

ACID Properties:

1. **Atomicity**:
    - **Definition**: A transaction is indivisible. It either completes in its entirety or not at all. If any part of the transaction fails, the entire transaction is rolled back, and the database state is unchanged.
    - **Example**: If a bank transfer involves debiting one account and crediting another, both operations must succeed. If one operation fails, neither is performed.
2. **Consistency**:

- **Definition**: A transaction brings the database from one valid state to another, preserving all integrity constraints (such as primary keys, foreign keys, and user-defined constraints). After a transaction completes, the database must be in a consistent state.

- **Example**: If a database enforces a rule that no employee can have a negative salary, a transaction that tries to set an employee's salary to a negative value must be rejected.

3. **Isolation**:

   - **Definition**: Transactions must be executed in isolation from one another. Even if transactions are executing concurrently, each transaction should execute as if it were the only transaction in the system. The intermediate states of a transaction must not be visible to other transactions.

   - **Example**: If two transactions are running concurrently, one modifying a bank account balance and the other trying to read it, the system ensures that either the first transaction is fully completed before the second one starts, or the second one sees the balance as if the first never occurred.

4. **Durability**:

   - **Definition**: Once a transaction has been committed, its changes to the database are permanent, even in the event of a system crash. The data must be stored in a way that survives failures (e.g., written to non-volatile storage like disk).

   - **Example**: After a successful bank transfer, even if the system crashes immediately after, the transaction's effect (i.e., the account balances) will persist after recovery.

A transaction must be in one of the following states:

• **Active,** the initial state; the transaction stays in this state while it is executing.

• Partially committed, after the final statement has been executed.

• **Failed,** after the discovery that normal execution can no longer proceed.

• Aborted, after the transaction has been rolled back and the database has been restored to its state prior to the start of the transaction.

• **Committed**, after successful completion.

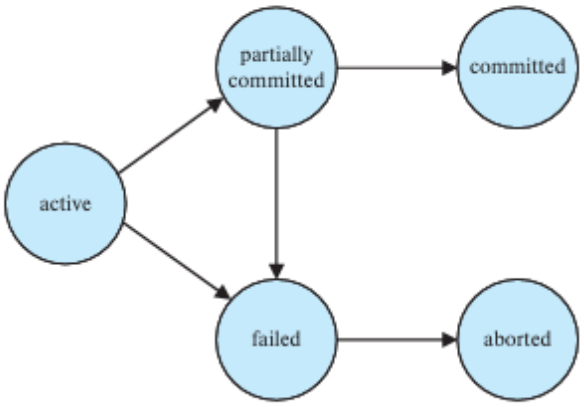


Figure 17.1 State diagram of a transaction.

A transaction can enter the **aborted state** under certain conditions. At this point, the system has two options:

1. **Restart the transaction**: This is possible if the transaction was aborted due to hardware or software errors, not due to internal logic errors. The restarted transaction is treated as a new transaction.

2. **Kill the transaction**: This happens when there's an internal logical error, bad input, or missing data that can only be corrected by rewriting the application program.

This part of the text is explaining how **conflicting instructions** in a schedule can be identified based on the types of operations (read and write) performed on the same data item. It discusses the four cases that determine whether the **order** of operations matters:

Four Cases to Consider:

1. **Case 1:** `I = read(Q), J = read(Q)`

    - **Explanation**: If both operations are **read** operations on the same data item **Q**, the order does not matter. The value of **Q** read by both transactions will be the same, regardless of the execution order.

- **Example**: If `T1` and `T2` both read **A**, it doesn't matter whether `T1` reads first or `T2` reads first, as both will get the same value of **A**.

2. **Case 2:** `I = read(Q), J = write(Q)`

   - **Explanation**: The **order** matters because the **write** operation will update the value of **Q**, affecting what the **read** operation sees. If the **read** happens before the **write**, it will read the old value; if the **write** happens before the **read**, the read will retrieve the new value.

   - **Example**: If `T1` reads **A** and `T2` writes **A**, the order determines whether `T1` sees the old or new value of **A**.

3. **Case 3:** `I = write(Q), J = read(Q)`

   - **Explanation**: Similar to Case 2, the **order** matters here too. If `T1` writes to **A** and `T2` reads **A**, the **read** will see the new value only if the **write** comes first. Otherwise, it will see the old value.

   - **Example**: If `T1` writes **A** and `T2` reads **A**, the result depends on whether the **write** or **read** comes first in the schedule.

4. **Case 4:** `I = write(Q), J = write(Q)`

   - **Explanation**: Here, both operations are **write** operations on the same data item. The order of execution doesn't affect the intermediate database states, but the final value of **Q** will be determined by the last **write** operation. If no other **write** follows, the order of the **write** operations directly affects the final value.

   - **Example**: If `T1` writes **A** and `T2` writes **A**, the final value of **A** will be whatever `T2` wrote if there are no further **writes**.

Concurrency Control in Database Systems

To ensure **isolation** when multiple transactions execute concurrently, the system must control how transactions interact with each other. This control is achieved using **concurrency-control schemes**. The most common ones are:

- **Two-phase locking**
- **Snapshot isolation**

These mechanisms prevent conflicts and preserve data integrity, even when multiple transactions access data simultaneously.

Lock-Based Protocols

One effective method to ensure isolation is to require **mutual exclusion** for accessing data. This means that when a transaction is interacting with a data item, no other transaction can modify it. This is done through **locks**.

Lock Modes:

1. **Shared Lock (S)**:
    - A transaction can **read** but **cannot write** the data item.
    - Multiple transactions can hold a shared lock on the same data item simultaneously.
2. **Exclusive Lock (X)**:
    - A transaction can **both read and write** the data item.
    - Only **one transaction** can hold an exclusive lock at a time for a particular data item.

Lock Compatibility Matrix:

The **compatibility** of locks determines when transactions can safely hold locks on the same data item. Here's the lock compatibility matrix:

Lock Type	Shared (S)	Exclusive (X)
Shared (S)	Yes	No

Exclusive (X)	No	No

- **Shared lock (S)** can coexist with other **shared locks** but not with **exclusive locks**.
- **Exclusive lock (X)** cannot coexist with any other lock.

This mechanism ensures that multiple transactions can read a data item concurrently but limits write access to a single transaction at a time.

Deadlock in Database Systems

A **deadlock** occurs when there is a set of transactions $\{T_0, T_1, ..., Tn\}$ where each transaction is waiting for another transaction to release a data item. Specifically, T_0 waits for a data item held by T_1, T_1 waits for a data item held by T_2, and so on, with Tn waiting for a data item held by T_0. In such a scenario, no transaction can proceed, and all are stuck in a waiting state.

Deadlock Handling Mechanisms

There are two main approaches to handle deadlocks:

1. **Deadlock Prevention**
2. **Deadlock Detection and Recovery**

---

1. Deadlock Prevention

Deadlock prevention protocols ensure that the system never enters a deadlock state by restricting how transactions acquire locks. These approaches guarantee that one of the necessary conditions for a deadlock (circular wait) is always violated.

Prevention Approaches:

1. **Locking All Data Items Before Execution**:
    - **Approach**: A transaction locks all the data items it needs before it starts execution. All items are locked at once or none at all.
    - **Disadvantages**:
        - Difficult to predict all data items a transaction will need.
        - Low data-item utilization, as many items may be locked but not used for a long time.
2. **Imposing Lock Ordering**:
    - **Approach**: Transactions acquire locks in a specific order, ensuring that no circular waits can occur.
        - **Partial Ordering**: For example, a tree protocol, which imposes an order on the data items.
        - **Total Ordering**: Locks are acquired in a strict, predefined order based on a global ordering of data items. Once a transaction locks an item, it cannot request locks on items that precede it in the order.
    - **Advantages**: Simple and prevents deadlocks as long as the order is respected.
    - **Disadvantages**: The transaction must know its data items beforehand.
3. **Preemption and Rollbacks**:
    - **Approach**: If a transaction requests a lock held by another transaction, the system may preempt (abort) one of the transactions and grant the lock to the other.
    - **Techniques**:
        - **Wait-Die Scheme**: A transaction with an older timestamp can wait for a younger transaction, while a younger transaction is rolled back (dies).

        - **Wound-Wait Scheme**: A younger transaction can wait for an older transaction, while the older transaction is preempted (wounded).

    - **Disadvantages**: May lead to unnecessary rollbacks.

4. **Lock Timeout**:

    - **Approach**: Transactions are allowed to wait for a lock for a specified period. If the lock is not granted within that time, the transaction is rolled back and restarted.

    - **Disadvantages**:

        - Hard to decide the right timeout duration.

        - May lead to unnecessary rollbacks or delays.

        - Starvation can occur if the same transactions are repeatedly rolled back.

---

2. Deadlock Detection and Recovery

If deadlock prevention is not used, deadlocks can occur, and a **detection and recovery scheme** must be implemented. The system detects deadlocks after they happen and recovers by aborting one or more transactions to break the deadlock.

Deadlock Detection:

- **Wait-for Graph**: A directed graph representing the waiting relationships between transactions.

    - **Vertices**: Represent transactions.

    - **Edges**: Represent the wait-for relationship (i.e., Ti → Tj means transaction Ti is waiting for transaction Tj).

- **Cycle Detection**: A deadlock exists if the wait-for graph contains a cycle. If a cycle is detected, the transactions involved in the cycle are deadlocked.

- **Example**:
    - Transactions T_1, T_2, T_3, and T_4 are waiting for each other in a cycle, indicating a deadlock.
- **When to Invoke Detection**:
    - If deadlocks occur frequently, the detection algorithm should run more frequently.
    - If deadlocks are rare, it can be run less often.

Deadlock Recovery:

Once a deadlock is detected, the system must recover by selecting a victim and rolling back one or more transactions.

1. **Victim Selection**:
    - Determine which transaction(s) to roll back to break the deadlock.
    - Criteria include:
        - **Transaction duration**: How long a transaction has been running.
        - **Data items used**: How many items the transaction has modified.
        - **Future data needs**: The number of items the transaction still needs.
        - **Rollback cost**: How many transactions will be affected by the rollback.
2. **Rollback**:
    - **Total Rollback**: Abort the transaction and restart it.
    - **Partial Rollback**: Undo only the actions necessary to release locks and break the deadlock. This requires maintaining information about the transaction's progress.
3. **Starvation Prevention**:

- To avoid the same transaction always being picked as the victim, the system can limit how many times a transaction can be rolled back.

Summary

1. **Deadlock Prevention**:
    - **Locking All Data Items Before Execution**
    - **Lock Ordering (Partial/Total)**
    - **Preemption and Rollbacks (Wait-Die/Wound-Wait)**
    - **Lock Timeout**
2. **Deadlock Detection and Recovery**:
    - **Wait-for Graph** for detecting cycles.
    - **Victim Selection** based on transaction cost factors.
    - **Rollback** (Total or Partial) and handling **Starvation**.

SQL Language Components

SQL (Structured Query Language) is the standard language used for managing and manipulating relational databases. It has various components and features to perform operations on data. Below are the key components of SQL:

---

1. SQL Language Parts

1. **DDL (Data Definition Language)**:

- Defines and modifies the database structure.
- Examples: `CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`.
- Used for creating tables, defining relationships, and modifying the database schema.

2. **DML (Data Manipulation Language)**:
   - Deals with data manipulation and querying.
   - Examples: `SELECT`, `INSERT`, `UPDATE`, `DELETE`.
   - Used to insert, update, delete, or retrieve data from the database.
3. **DCL (Data Control Language)**:
   - Deals with permissions and access control.
   - Examples: `GRANT`, `REVOKE`.
   - Used to give and remove user permissions.
4. **TCL (Transaction Control Language)**:
   - Deals with transactions in databases.
   - Examples: `COMMIT`, `ROLLBACK`, `SAVEPOINT`, `SET TRANSACTION`.
   - Used to manage changes made by DML commands and handle transactions.

---

2. SQL Data Types

SQL supports a variety of data types, categorized as follows:

1. **Numeric Data Types**:

- `INT` or `INTEGER`: Whole numbers.
- `DECIMAL(p, s)` or `NUMERIC(p, s)`: Fixed-point numbers (p is precision, s is scale).
- `FLOAT`, `REAL`, `DOUBLE PRECISION`: Floating-point numbers for approximations.

2. **Character Data Types**:
   - `CHAR(n)` or `CHARACTER(n)`: Fixed-length character strings (n is the length).
   - `VARCHAR(n)` or `CHARACTER VARYING(n)`: Variable-length character strings.
   - `TEXT`: Variable-length character strings, typically used for large text data.
3. **Date and Time Data Types**:
   - `DATE`: Date value (YYYY-MM-DD).
   - `TIME`: Time value (HH:MM:SS).
   - `DATETIME`: Date and time together (YYYY-MM-DD HH:MM:SS).
   - `TIMESTAMP`: Date and time with timezone information.
   - `INTERVAL`: Time interval between two date/time values.
4. **Boolean Data Type**:
   - `BOOLEAN`: Stores `TRUE`, `FALSE`, or `NULL`.
5. **Binary Data Types**:
   - `BLOB`: Binary large object for storing large binary data.
   - `BYTEA`: For storing raw binary data.

6. **Other Data Types**:
    - `ENUM`: A predefined set of values.
    - `UUID`: Universally unique identifier.

---

3. SQL Operators

1. **Comparison Operators**:
    - `=`: Equal to.
    - `!=`, `<>`: Not equal to.
    - `>`: Greater than.
    - `<`: Less than.
    - `>=`: Greater than or equal to.
    - `<=`: Less than or equal to.
    - `BETWEEN`: Checks if a value is within a range.
    - `IN`: Checks if a value is within a list.
    - `LIKE`: Pattern matching for string values.
    - `IS NULL`: Checks for NULL values.
2. **Logical Operators**:
    - `AND`: Combines two conditions, both must be true.
    - `OR`: Combines two conditions, at least one must be true.

- `NOT`: Negates a condition.

3. **Mathematical Operators**:
   - `+`: Addition.
   - `-`: Subtraction.
   - `*`: Multiplication.
   - `/`: Division.
   - `%`: Modulo (remainder of division).
4. **String Operators**:
   - `CONCAT()`: Combines multiple strings into one.
   - `||`: Concatenation (for some SQL dialects like PostgreSQL).
   - `SUBSTRING()`: Extracts a substring from a string.
   - `TRIM()`: Removes leading/trailing spaces.
   - `LENGTH()`: Returns the length of a string.

---

4. SQL Match Operations

Match operations allow you to perform pattern matching in SQL queries, often used with the `WHERE` clause.

1. **LIKE**:
   - Used for pattern matching in string values.
   - Supports wildcard characters:

- `%` (percentage): Matches any sequence of characters (including no characters).
- `_` (underscore): Matches exactly one character.

Example:

```
 SELECT * FROM employees WHERE name LIKE 'J%';  -- Names starting with 'J'
```

2. **REGEXP** (or `RLIKE` in some databases):
   - Supports regular expressions for advanced pattern matching.

Example:

```
   SELECT    *    FROM    users    WHERE    email    REGEXP '^[A-Za-z0-9]+@[A-Za-z]+\.[A-Za-z]{2,}$';
```

---

5. SQL Aggregate Functions

Aggregate functions are used to perform calculations on multiple rows of data and return a single result. They are often used with `GROUP BY` to group results by one or more columns.

1. **COUNT()**:
   - Returns the number of rows.

Example:

```
SELECT COUNT(*) FROM employees;
```

2. **SUM()**:
    - Returns the sum of a numeric column.

Example:

```
SELECT SUM(salary) FROM employees;
```

    -
3. **AVG()**:
    - Returns the average of a numeric column.

Example:

```
SELECT AVG(age) FROM students;
```

    -
4. **MIN()**:
    - Returns the minimum value of a column.

Example:

```
SELECT MIN(price) FROM products;
```

    -
5. **MAX()**:
    - Returns the maximum value of a column.

Example:

```
SELECT MAX(price) FROM products;
```

    -
6. **GROUP_CONCAT()** (in MySQL) or **STRING_AGG()** (in PostgreSQL):

    - Concatenates values from multiple rows into a single string.

Example:

```
SELECT GROUP_CONCAT(name) FROM employees;
```

    -

---

6. SQL Clauses

- **SELECT**: Retrieves data from a database.
    - Example: `SELECT * FROM employees;`
- **WHERE**: Filters records based on specified conditions.
    - Example: `SELECT * FROM employees WHERE salary > 50000;`
- **ORDER BY**: Sorts the result set.
    - Example: `SELECT * FROM employees ORDER BY salary DESC;`
- **GROUP BY**: Groups rows that have the same values in specified columns.
    - Example: `SELECT department, AVG(salary) FROM employees GROUP BY department;`
- **HAVING**: Filters records after a `GROUP BY` clause.
    - Example: `SELECT department, AVG(salary) FROM employees GROUP BY department HAVING AVG(salary) > 50000;`
- **LIMIT**: Limits the number of rows returned.

- Example: `SELECT * FROM employees LIMIT 10;`

select *
from (**select** R_1 **from** r)
natural join
(**select** R_2 **from** r)

This is stated more succinctly in the relational algebra as:

$$\Pi_{R_1}(r) \bowtie \Pi_{R_2}(r) = r$$

1. INNER JOIN and OUTER JOIN Types

An INNER JOIN is one of the most common types of joins in SQL, which returns only the rows where there is a match in both tables being joined. In other words, it retrieves records that have matching values in both tables. While the term "INNER JOIN" generally refers to this basic operation, there are variations or subtypes depending on the context or SQL dialect used.

Here's an overview of the INNER JOIN and how it can be used:

1. Basic INNER JOIN:

The basic `INNER JOIN` retrieves only the rows from both tables where the specified condition (usually a match on a common column) is true.

Syntax:

```
SELECT columns

FROM table1

INNER JOIN table2

ON table1.column_name = table2.column_name;
```

Example:

Suppose we have two tables: `Employees` and `Departments`.

Employees Table:

emp_id	name	dept_id
1	Alice	101
2	Bob	102
3	Charlie	101
4	David	103

Departments Table:

dept_id	dept_name
101	HR
102	IT
104	Marketing

Using an `INNER JOIN` to get the names of employees along with their department names:

```
SELECT Employees.name, Departments.dept_name
FROM Employees
INNER JOIN Departments
ON Employees.dept_id = Departments.dept_id;
```

Result:

name	dept_name
Alice	HR
Bob	IT
Charlie	HR

- `David` is not included because their `dept_id` (103) does not match any in the `Departments` table.
- The department `Marketing` (dept_id 104) is not shown because no employee has that department ID.

**2. SELF JOIN (Self-Referencing INNER JOIN):

A SELF JOIN is when a table is joined with itself. It's often used when you have hierarchical data or need to compare rows within the same table.

Syntax:

```
SELECT a.columns, b.columns
FROM table a, table b
WHERE a.column_name = b.column_name;
```

Example:
Consider an `Employees` table where employees report to other employees (supervisor and subordinate).

Employees Table:

emp_id	name	manager_id
1	Alice	NULL

2	Bob	1
3	Charlie	1
4	David	2

To find the names of employees and their managers, we can join the `Employees` table with itself:

```
SELECT e1.name AS Employee, e2.name AS Manager
FROM Employees e1
INNER JOIN Employees e2
ON e1.manager_id = e2.emp_id;
```

Result:

Employee	Manager
Bob	Alice
Charlie	Alice
David	Bob

Here, `e1` is the employee and `e2` is their manager.

3. CROSS JOIN (Cross Join with INNER JOIN Logic)

Although `CROSS JOIN` is not exactly the same as `INNER JOIN`, sometimes the results from a `CROSS JOIN` can be processed similarly to a form of "unlimited matching" that may resemble a Cartesian product or full join, where every row from one table is paired with every row of the other table.

Example:

```
SELECT *
```

```
FROM Employees

CROSS JOIN Departments;
```

This will produce a result where every employee is paired with every department, leading to a Cartesian product. However, when combined with filters or conditions, it might mimic behavior of INNER JOIN in some contexts.

OUTER JOIN Types:

An **Outer Join** returns all the rows from one table and the matched rows from the other. If there is no match, the result will contain `NULL` for columns of the table that does not have a matching row.

There are three main types of **Outer Joins**:

LEFT OUTER JOIN: Returns all rows from the left table, and the matched rows from the right table. If no match, `NULL` values are returned for right table columns.

Example:

```
SELECT emp_id, name, dept_name
FROM Employees
LEFT OUTER JOIN Departments
ON Employees.dept_id = Departments.dept_id;
```

1. **Result:**

emp_id	name	dept_name
1	Alice	HR
2	Bob	IT
3	Charlie	HR

2.

(If `dept_id` 103 existed in Employees, that would also show with `NULL` in `dept_name`)

RIGHT OUTER JOIN: Returns all rows from the right table, and the matched rows from the left table. If no match, `NULL` values are returned for left table columns.

Example:

```
SELECT emp_id, name, dept_name
FROM Employees
RIGHT OUTER JOIN Departments
ON Employees.dept_id = Departments.dept_id;
```

3. **Result:**

emp_id	name	dept_name
1	Alice	HR
2	Bob	IT
3	Charlie	HR
NULL	NULL	Sales

4.

FULL OUTER JOIN: Returns all rows when there is a match in either left or right table. Non-matching rows will have `NULL` in the columns where there is no match.

Example:

```
SELECT emp_id, name, dept_name
FROM Employees
FULL OUTER JOIN Departments
ON Employees.dept_id = Departments.dept_id;
```

5. **Result:**

emp_id	name	dept_name
1	Alice	HR

2	Bob	IT
3	Charlie	HR
NULL	NULL	Sales

6.

---

2. NULL Constant in SQL:

A `NULL` in SQL is a special marker used to indicate that a data value does not exist in the database. It's not the same as an empty string or `0`; it means the value is unknown or missing.

Example of NULL:

```
SELECT name, salary FROM Employees WHERE salary IS NULL;
```

This query would return all employees who have no salary assigned (i.e., their salary is `NULL`).

---

3. Weak Entity Set Representation in ER Model:

A **weak entity** does not have a primary key of its own and relies on a **strong entity** for identification. A weak entity always has a **partial key** and depends on the strong entity through a **foreign key relationship**.

- **Weak Entity**: An entity that cannot be uniquely identified by its own attributes.
- **Strong Entity**: An entity that can be uniquely identified by its own attributes.

Weak Entity Representation:

- **Symbol**: Double rectangle
- **Relationship**: Double diamond

- **Participation**: Total participation (always related to a strong entity)

Example:
In a scenario where `Order` is a weak entity dependent on `Customer`:

- `Order` entity may have attributes `order_id`, `order_date`, but it cannot be uniquely identified without referencing `Customer`.

---

4. Unified Modeling Language (UML):

UML is a standardized modeling language used to specify, visualize, construct, and document the artifacts of a software system. It includes several types of diagrams for different purposes.

Main Parts of UML:

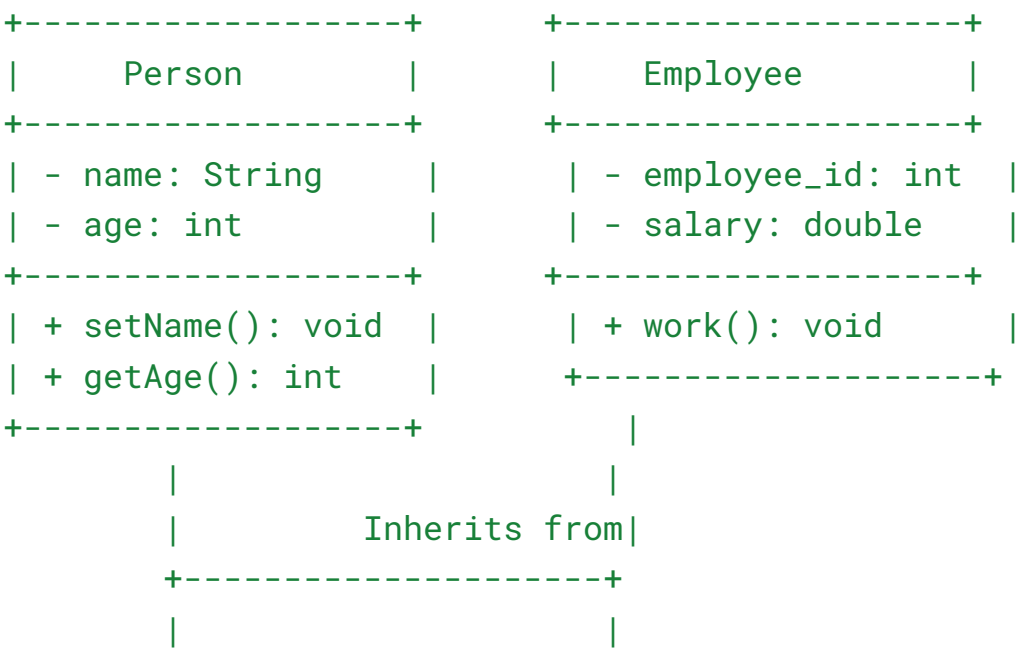
1. **Structure Diagrams**: Describe the static structure of a system.
    - **Class Diagram**: Represents the classes, their attributes, methods, and relationships (e.g., inheritance, association).
    - **Component Diagram**: Describes how the system is divided into components (modules, etc.).
    - **Deployment Diagram**: Describes the physical deployment of artifacts and their relationship.
    - **Object Diagram**: Shows an instance of a class diagram, representing the state of the system at a particular point in time.
    - **Package Diagram**: Organizes classes into packages and shows their dependencies.
2. **Behavior Diagrams**: Describe the dynamic behavior of the system.
    - **Use Case Diagram**: Represents the interactions between users (actors) and the system.

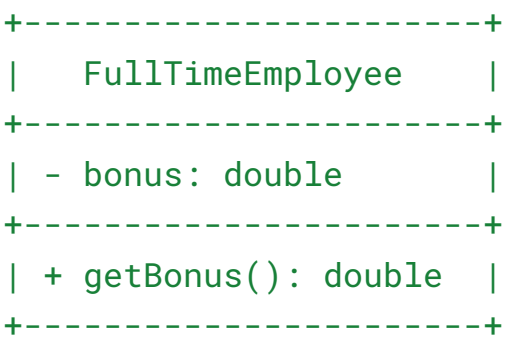
- **Sequence Diagram**: Shows how objects interact in a particular scenario in a sequence of messages.
- **Collaboration Diagram**: Similar to sequence diagram but focuses on relationships between objects.
- **State Diagram**: Describes the states of an object and how it transitions between those states.
- **Activity Diagram**: Represents workflows and the flow of control in the system.

3. **Interaction Diagrams**: A subset of behavior diagrams used to show how objects interact.
   - **Sequence Diagrams**: Show how objects interact over time.
   - **Communication Diagrams**: Focus on the interactions between objects.
   - **Timing Diagrams**: Depict the time-based behavior of objects.

UML Example - Class Diagram:

Here's a simple class diagram for a `Person` and `Employee` class:





- `Person` is a general class with `name` and `age`.
- `Employee` inherits from `Person` and adds attributes specific to employees (`employee_id`, `salary`).
- `FullTimeEmployee` inherits from `Employee` and adds `bonus`.